

noun to describe the thing being acted on. For example, the names `refreshConnection()`, `setPassword()`, and `extract_version()` clarify what the function does and to what.

You might not need a noun for methods that are part of a class or module. A `reset()` method in a `SatelliteConnection` class or an `open()` function in the `webbrowser` module already provides the necessary context. You can tell that a satellite connection is the item being reset and that a web browser is the item being opened.

It's better to use long, descriptive names rather than an acronym or a name that's too short. A mathematician might immediately understand that a function named `gcd()` returns the greatest common denominator of two numbers, but everyone else would find `getGreatestCommonDenominator()` more informative.

Remember not to use any of Python's built-in function or module names, such as `all`, `any`, `date`, `email`, `file`, `format`, `hash`, `id`, `input`, `list`, `min`, `max`, `object`, `open`, `random`, `set`, `str`, `sum`, `test`, and `type`.

Function Size Trade-Offs

Some programmers say that functions should be as short as possible and no longer than what can fit on a single screen. A function that is only a dozen lines long is relatively easy to understand, at least compared to one that is hundreds of lines long. But making functions shorter by splitting up their code into multiple smaller functions can also have its downsides. Let's look at some of the advantages of small functions:

- The function's code is easier to understand.
- The function likely requires fewer parameters.
- The function is less likely to have side effects, as described in “Functional Programming” on page 172.
- The function is easier to test and debug.
- The function likely raises fewer different kinds of exceptions.

But there are also some disadvantages to short functions: